DIY Challenge Next Steps Checklist

Implementation order based on the SAD in `docs/diy-sad.html`. Keep this updated as the stack evolves.

1. Safety and Control Foundation

- Create `diy_cmd_vel_mux` to arbitrate joystick, autonomous, and E-stop states.
- Add `diy_estop_controller` with wired/wireless stop handling and watchdog behavior.
- Ensure the motor driver only consumes one safe topic, ideally `/cmd_vel_safe`.

2. Bringup Baseline

- Keep the reusable legacy motor package as a bootstrap only.
- Use `challenge_bringup` as the primary launch entry point.
- Confirm joystick mode, driver mode, and later autonomous mode can be launched cleanly.

3. Runtime Localization

- Integrate FAST-LIO2 for LiDAR-IMU odometry.
- Wire `robot_localization` with EKF1 for local odom.
- Add EKF2 with RTK GNSS and `navsat_transform` for global pose.
- Verify the TF tree early so frame issues are caught before later work.

4. Offline Mapping Pipeline

- Use LIO-SAM for prior map generation.
- Add rosbag recording and replay workflows for calibration and debugging.
- Generate the prior map, 2D map, centerline waypoints, and zone config files.

5. Perception and Behavior

- Implement the PCL obstacle classifier.
- Detect chimes, walls, movable objects, tunnels, and passage zones.
- Add behavior modes for push-through, navigate-around, and blind-drive.

6. Mission Layer

- Implement the state machine: WAIT, NAVIGATE, ZONE_ENTRY, BEHAVIOR_SWITCH, LAP_COMPLETE, STOP.
- Connect mission flow to Nav2 goals and zone boundaries.
- Make the logic lap-aware so lap 2 can handle moved obstacles correctly.

7. Calibration and Hardening

- Calibrate IMU, LiDAR-IMU extrinsics, and camera extrinsics.
- Tune costmaps, controller gains, and RTK correction timing.
- Profile Jetson load and reduce rates if needed.

Suggested First Coding Task

Build `diy_cmd_vel_mux` plus the E-stop interface first. That creates the safe control backbone for every later subsystem.